

Documentation

ShopAssistAI: Building Custom Chatbot with OpenAI

Date: 03/08/2025

Name: Munish Gupta

Table of Contents

SHOPASSISTAI: BUILDING CUSTOM CHATBOT WITH LLMs.....	1
<u>1. INTRODUCTION.....</u>	<u>3</u>
<u>2. SYSTEM REQUIREMENTS.....</u>	<u>3</u>
<u>3. SYSTEM FUNCTIONALITIES</u>	<u>3</u>
<u>4 EXTRACT USER REQUIREMENTS</u>	<u>3</u>
<u>5 BUILD THE USER PROFILE</u>	<u>4</u>
<u>6. SYSTEM ARCHITECTURE</u>	<u>4</u>
SYSTEM ARCHITECTURE LAYERS	4
<u>7. IMPLEMENTATION DETAILS.....</u>	<u>5</u>
<u>8. CONCLUSION</u>	<u>5</u>
<u>9. APPENDIX – I.....</u>	<u>6</u>

1. Introduction

ShopAssistAI is an intuitive, web-based conversational platform that helps users discover the ideal laptop tailored to their specific needs. Powered by Generative AI, it engages users in a guided dialogue to understand their preferences and then recommends the most suitable options from a curated laptop database

2. System Requirements

- **Front-End:** HTML, CSS, Javascript
- **Back-End:** Python, Flask web framework
- **AI Engine:** OpenAI's API for conversation generation and moderation
- **Database:** Laptop_data.csv

3. System functionalities

- **User Interface:** ShopAssistAI features a clean, user-friendly web interface that enables seamless interaction with its conversational AI assistant.
- **Conversational AI:** At the heart of ShopAssistAI is a powerful conversational AI, built on OpenAI's chat model. It engages users through dynamic dialogue, asking insightful questions to understand their laptop requirements.
- **User Input Moderation:** At the heart of ShopAssistAI is a powerful conversational AI, built on OpenAI's chat model. It engages users through dynamic dialogue, asking insightful questions to understand their laptop requirements.
- **User Profile Extraction:** As the conversation progresses, the AI assistant extracts key details—such as budget, screen size, and performance needs—to construct a personalized user profile. This is facilitated by OpenAI's function calling mechanism, which transforms user requirements from natural language into structured JSON format.

4 Extract User Requirements

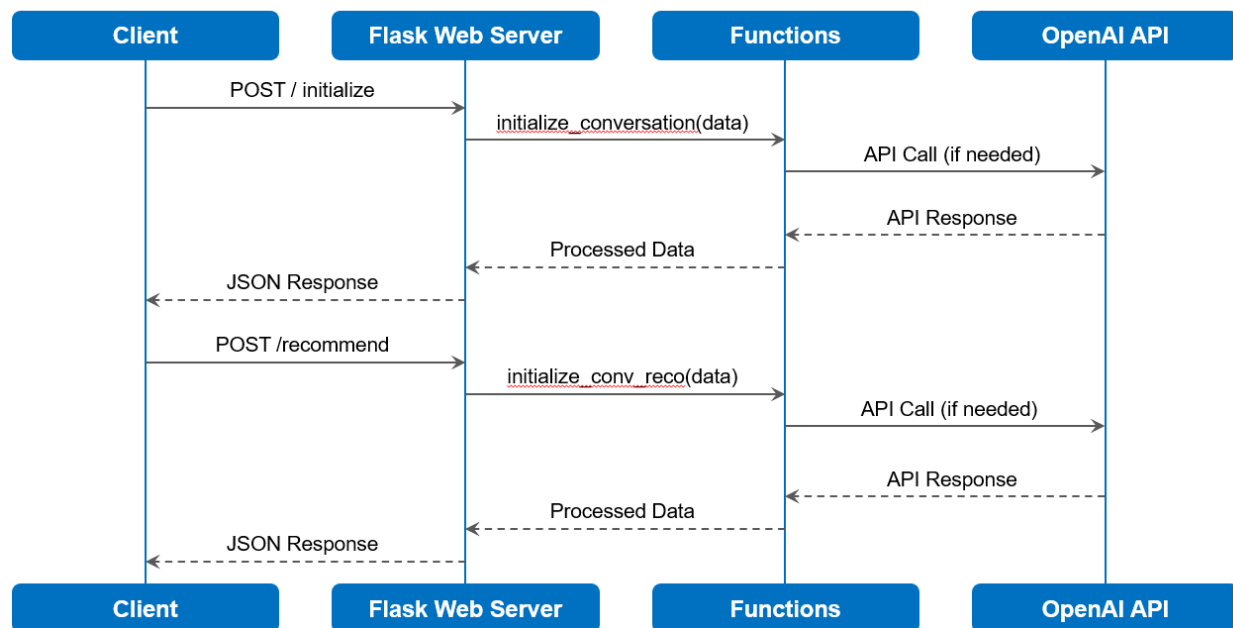
- ShopAssistAI leverages OpenAI's chat-completion endpoint to generate dynamic responses and extract user preferences during the conversation. By sending structured prompts along with the conversation history, the system captures user-expressed requirements naturally within the dialogue flow.
- Although OpenAI's API doesn't natively support direct text-to-JSON conversion, ShopAssistAI employs OpenAI's function calling mechanism to transform user input into structured key-value pairs. This technique enables accurate extraction of laptop preferences such as budget, screen size, and performance needs.

5 Build the User Profile

- The extracted key-value pairs can then be used to construct a user profile dictionary in JSON format. This dictionary would represent the user's preferences for various laptop attributes.

6. System Architecture

ShopAssistAI is built on a client-server model. Users interact through a web interface hosted on a server running a Flask-based application. This backend communicates with OpenAI's API to generate conversational responses and perform content moderation. Additionally, it connects to an external database to retrieve and compare laptop specifications, enabling personalized recommendations.



System Architecture Layers

1. Client (Frontend)

- Role:** Presents a user interface (web page or app) for users to enter prompts and displays responses from the AI.
- Request:** Sends HTTP POST request with user input (prompt) in JSON format (e.g., `{"prompt": "Your text"}`).
- Response:** Receives AI-generated text in JSON, updates the UI dynamically.

2. Flask Server (Backend)

- **Role:** Accepts client requests, performs validation, authentication, logging, and routes input to the appropriate function modules.
- **Request:** Receives HTTP POST/GET data from the client. Extracts user prompt from JSON payload.
- **Response:** Sends AI's output as a JSON response.

3. Function Modules (Business Logic)

- **Role:** Contains custom logic for processing input, error handling, or orchestrating multi-step flows. Calls the OpenAI API with the formatted data.
- **Request:** Receives Python objects (e.g., the prompt string) from Flask routes.
- **Response:** Returns processed results from OpenAI or error info back to the Flask route handler.

4. OpenAI API (External Service)

- **Role:** Receives API calls from the backend with prompts, processes them with LLMs, returns generated text or output in structured format.
- **Request:** Flask uses HTTP POST to send a JSON body (containing prompt, model, parameters) to the OpenAI endpoint (e.g., `/v1/completions`).
- **Response:** Returns JSON object with generated content, error messages, or structured data.

7. Implementation Details

The Flask application utilizes various functionalities:

- **Routing:** Maps user requests to appropriate functions based on URLs.
- **Conversation Management:** Handles conversation initiation, response generation through OpenAI's chat model, and conversation history maintenance.
- **User Input Processing:** Captures user input, performs moderation checks, and extracts user profiles from conversation history (using Open AI Function calling it converts the user input string to JSON object).
- **Recommendation Logic:** Compares user profiles with laptop data, validates recommendations, and generates recommendation text.

8. Conclusion

ShopAssistAI offers an intuitive and engaging platform that helps users effortlessly discover the ideal laptop. By harnessing AI-driven conversations and personalized recommendations, it streamlines the decision-making process for buyers.

Future enhancements may include:

- **Advanced AI Integration:** Incorporating a broader range of AI models to enable more sophisticated and natural interactions, potentially utilizing OpenAI's fine-tuning capabilities to specialize in laptop recommendations.
- **Product Range Expansion:** Extending the recommendation engine to cover additional categories of electronic devices.
- **Feedback-Driven Improvement:** Introducing a user feedback system to refine recommendation accuracy over time, possibly leveraging reinforcement learning to adapt and improve based on user interactions.

9. Appendix – I

User output example screenshot:

1. Screenshot A:

Shopping Assistant

Hello! Welcome to the laptop recommendation assistant. Please share your requirements so that I can help you find the best laptop that suits your needs.

i am looking for laptop for developer

i am looking for laptop for developer

Great! As a developer, you likely require a laptop that can handle coding, testing, and running multiple applications simultaneously. This means you would need a laptop with good processing speed and multitasking capabilities. Could you please share more details about your specific requirements or any preferences you have while working as a developer?

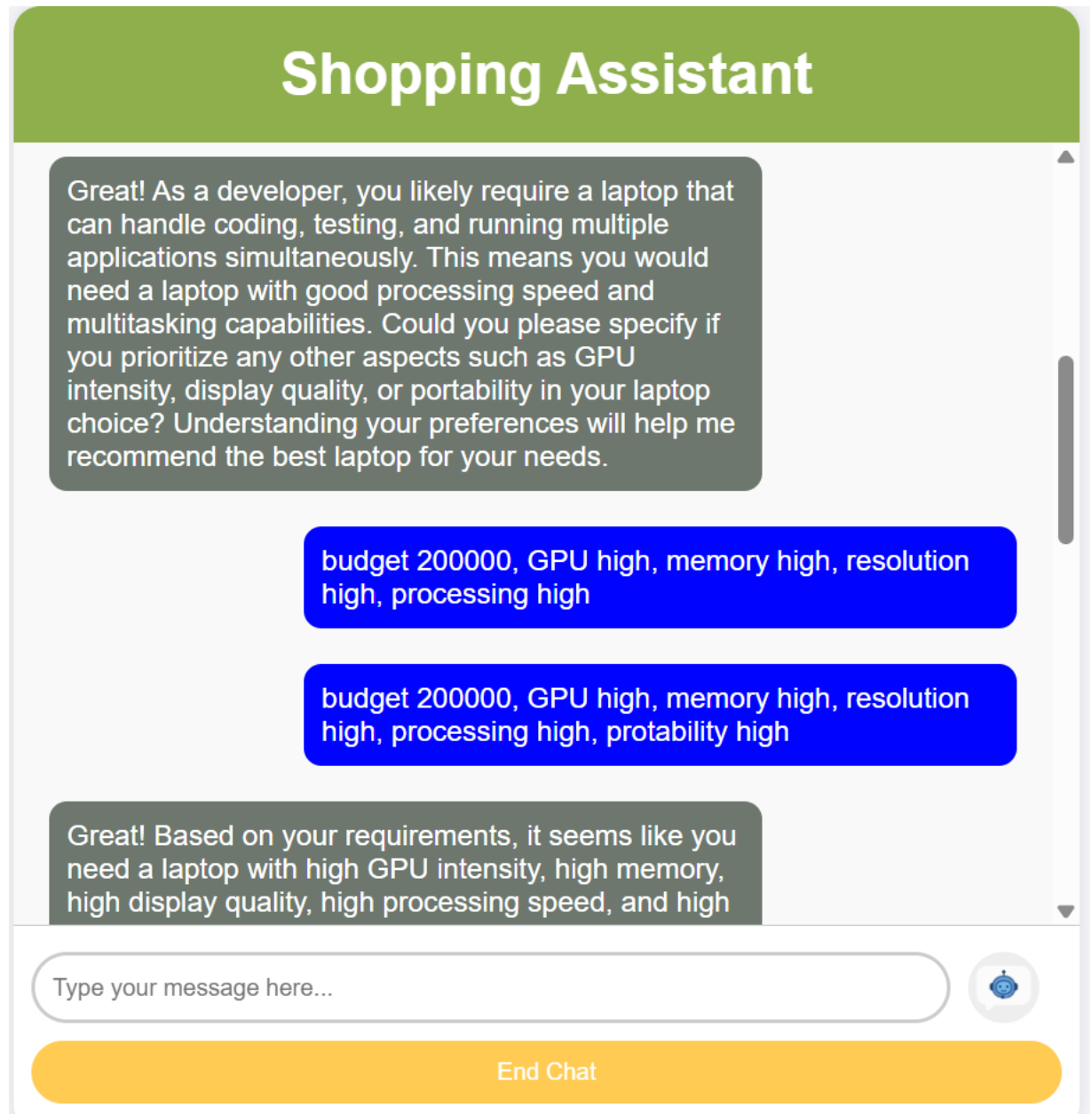
Great! As a developer, you likely require a laptop that

Type your message here...

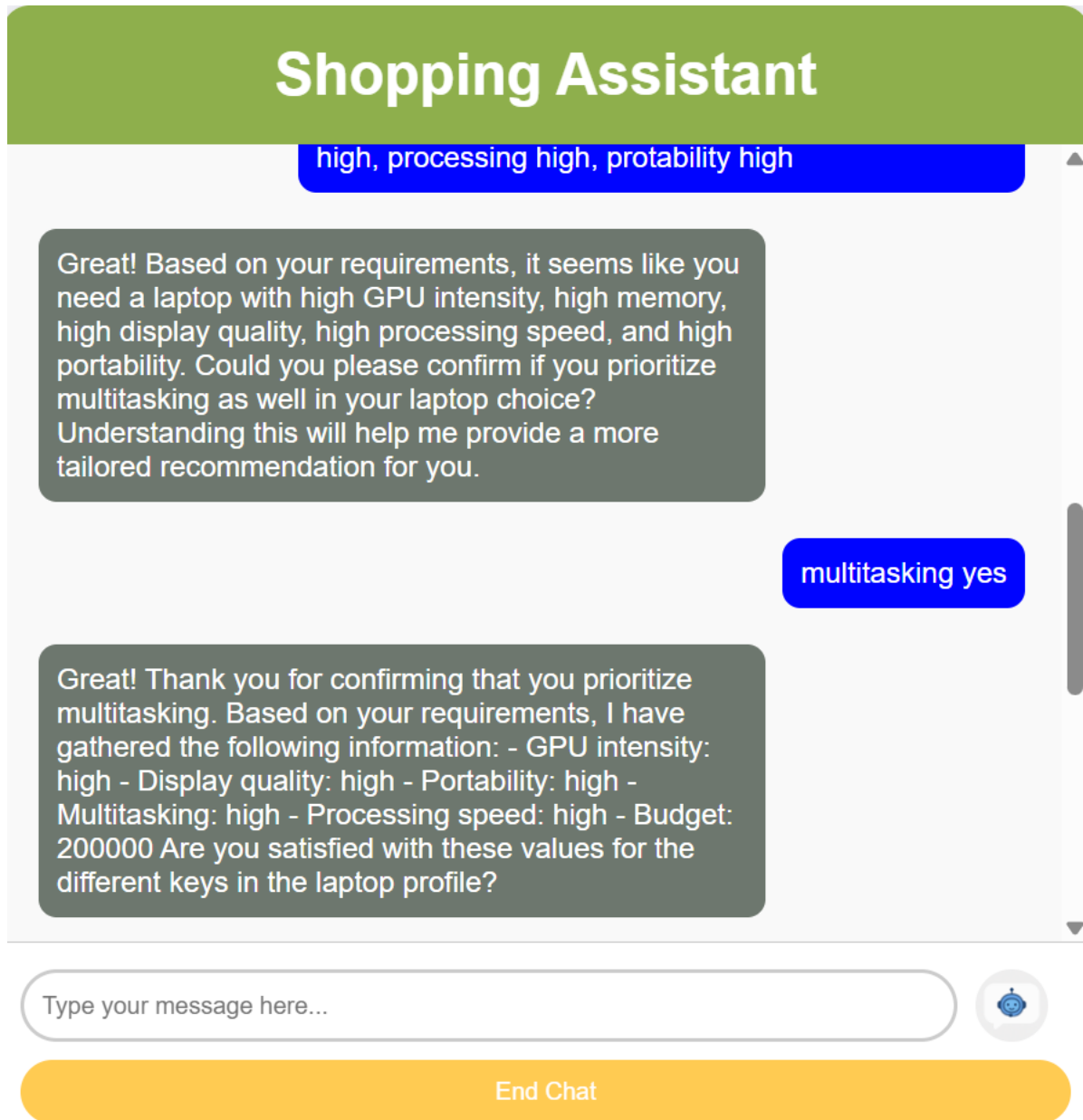


End Chat

2. Screenshot B:



3. Screenshot C:



4. Screenshot D:

Shopping Assistant

Thank you for providing all the information. Kindly wait, while I fetch the products:

Sure, here is a brief summary of each laptop in the catalogue, starting with the most expensive one: 1. ASUS ZenBook Pro: Intel Core i9 processor, 64GB RAM, 15.6" OLED display (3840x2160), NVIDIA RTX graphics card, Windows 10, 7 hours battery life, 1.8 kg, NanoEdge Display, 2 years warranty, Price: Rs 200,000 2. Dell XPS 15: Intel Core i9 processor, 32GB RAM, 15.6" OLED display (3840x2160), NVIDIA GTX graphics card, Windows 11, 8 hours battery life, 1.8 kg, Adobe RGB color gamut, 3 years warranty, Price: Rs 180,000 3. Lenovo ThinkPad: Ryzen 7 processor, 16GB RAM, 14" IPS display (2560x1440), NVIDIA GTX graphics card, Linux OS, 6 hours battery life, 1.6 kg, Backlit Keyboard, 3 years warranty, Price: Rs 60,000 How can I assist you further with this information?

Type your message here...



End Chat

